



Spark

Social App

Technical Documentation

Generated: May 7, 2026

10

Features

19

API Routes

6

Socket Events

11

Pages

This document covers the complete Spark social app - built with Node.js, Supabase Realtime, and a plain-HTML frontend. Re-run `node generate_docs.js` to update.

Contents

1. Tech Stack
2. How to Run
3. Features
4. API Endpoints
5. Socket.io Events
6. File Structure
7. Credentials & Config
8. Data Models
9. Reasonix Setup
10. Supabase Local Setup

1. Tech Stack

Layer	Technology	Notes
Backend	Node.js + Express 4	REST API + static file serving
Real-time	Supabase Realtime	Instant messaging via DB subscriptions
Database	Supabase (PostgreSQL)	Cloud Postgres + Realtime + Storage
Auth	JWT + bcryptjs	7-day tokens stored in localStorage
File Uploads	Multer	Profile photos (5 MB), chat files (10 MB)
Frontend	HTML + CSS + Vanilla JS	No framework - zero build step

2. How to Run

- Prerequisites**
- Node.js v16 or later (nodejs.org)
 - A terminal / command prompt

First-time setup Open a terminal in the project folder and run:

```
cd "C:\Users\andre\Documents\Deepseek\Code App"
npm install
```

Start the server

```
node server.js
```

- Open in browser**
- Navigate to `http://localhost:3000`
 - Register an account, complete your profile, then start browsing.
 - Admin panel: `http://localhost:3000/admin.html`

Always use `http://localhost:3000` - opening the HTML file directly via `file://` will not work.

3. Features

3.1 Authentication

`public/index.html` · `server.js`

- Register with username, email, and password (bcrypt-hashed - never stored plain)
- Login returns a 7-day JWT stored in localStorage
- Forgot password - 2-step: verify email, then set a new password (no email server needed)
- Change password on the Profile page (current password required)
- Clear error shown when app is opened as `file://` instead of `http://localhost:3000`

3.2 Profile

`public/profile.html` · `server.js`

- Upload a profile photo (JPEG / PNG, max 5 MB) shown as a circular thumbnail
- Set display name, age, gender, and bio (300-character counter)
- Email shown read-only
- Navbar chip shows name and avatar on all pages; falls back to initial letter

3.3 Discover / Browse

[public/browse.html](#) · [server.js](#)

- Responsive card grid of all members who completed their profile
- 88 px circular photo with a live green/grey online status dot
- Shows name, age, gender, bio, and last-seen time
- Click any card or the Message button to open a chat with that person
- Your own card is included and always shows Online while you are logged in

3.4 Real-Time Chat

[public/chat.html](#) · [server.js](#)

- Instant messaging via Socket.io - no page refresh needed
- Conversation sidebar with last-message preview and per-conversation unread badge
- Emoji picker with 7 categories: Smileys, Hearts, Hands, Nature, Food, Activity, Symbols
- File & image sharing up to 10 MB; images shown inline, other files as download links
- Preview bar before sending with X to cancel
- Edit own messages - hover to reveal button, confirm with Enter; shows "(edited)"
- Delete own messages - removed instantly for both sender and receiver

3.5 Online / Offline Presence

[public/global.js](#) · [server.js](#) · [browse.html](#) · [chat.html](#)

- Green = online, grey = offline on every member card and in the chat header
- Updates in real time via `user_status` socket event - no page reload needed
- Shows "Online" when connected, "Offline · Xm ago" after disconnect
- Global: user stays online while on Browse, Profile, or any other authenticated page
- Last-seen timestamp saved to `db.json` on socket disconnect

3.6 Unread Badges & Browser Notifications

[public/chat.html](#) · [public/global.js](#)

- Red badge counter per conversation in the sidebar
- Nav-bar badge showing total unread count across all conversations
- Desktop browser notifications when a message arrives (requires user permission)
- Clicking a notification opens the relevant chat directly
- Browser tab title flashes with the sender name when the window is not focused

3.7 Admin Panel

[public/admin.html](#) · [server.js](#)

- Separate password-protected dashboard at `/admin.html`
- Stats bar: total users, complete profiles, users with photos, total messages
- Users tab: searchable table with all fields and a delete button per row
- Messages tab: full log with sender, receiver, content, timestamp; delete per row
- Refresh button reloads live data without a page reload
- One-click link to Error Logs page

3.8 Error Logging

[logger.js](#) · [server.js](#) · [public/logs.html](#)

- All uncaught exceptions and unhandled promise rejections are written to `error.log`
- Route errors, failed login attempts, and admin login events are also logged
- Admin-only `/api/logs` endpoint to read the log file
- Admin-only `DELETE /api/logs` to clear the log
- Dedicated log viewer at `/logs.html` with dark terminal theme
- Auto-refresh every 5 seconds - no page reload needed
- Color-coded entries: INFO (green), WARN (amber), ERROR (red) with full stack traces

3.9 Contact Form

[public/profile.html](#) · [server.js](#)

- Send Message button on the Profile page opens a modal contact form
- Fields: Your Name, Subject, Message (1000-character limit with counter)
- Messages are sent via Formspree API to the developer email
- Success confirmation on submission, auto-closes after 2 seconds

- Accessible to all authenticated users from the Profile page

4. API Endpoints

All routes are relative to `http://localhost:3000`. Auth = Bearer token required. Admin = admin JWT required.

Method	Route	Auth?	Description
POST	<code>/api/register</code>	No	Create a new account
POST	<code>/api/login</code>	No	Login - returns JWT + userId
POST	<code>/api/forgot-password</code>	No	Verify email exists, return reset token (15 min TTL)
POST	<code>/api/reset-password</code>	No	Set new password using reset token
PUT	<code>/api/change-password</code>	Yes	Change password (current password required)
PUT	<code>/api/profile</code>	Yes	Update profile fields + optional photo upload
GET	<code>/api/profile/me</code>	Yes	Get own profile data
GET	<code>/api/members</code>	Yes	List all members with complete profiles
GET	<code>/api/messages/:otherId</code>	Yes	Message history with another user
GET	<code>/api/conversations</code>	Yes	Conversation list with last-message preview
POST	<code>/api/chat/upload</code>	Yes	Upload a file for use in chat
POST	<code>/api/admin/login</code>	No	Admin login - returns admin JWT
GET	<code>/api/admin/users</code>	Admin	All users (passwords excluded)
GET	<code>/api/admin/messages</code>	Admin	Full message log with sender/receiver names
GET	<code>/api/admin/stats</code>	Admin	Aggregate stats (users, messages, etc.)
DELETE	<code>/api/admin/users/:id</code>	Admin	Delete user and all their messages
DELETE	<code>/api/admin/messages/:id</code>	Admin	Delete a single message
GET	<code>/api/logs</code>	Admin	Read the server error log file
DELETE	<code>/api/logs</code>	Admin	Clear the error log file

5. Socket.io Events

All socket connections pass `auth: { token }` in the handshake on port 3000.

Event	Direction	Payload Fields	Description
send_message	Client -> Server	receiverId, content, type, fileName	Send message; server saves + routes it
new_message	Server -> Client	id, sender_id, receiver_id, content, type, sender_name	Delivered to sender and receiver
edit_message	Client -> Server	messageId, content	Edit own message
message_edited	Server -> Client	messageId, content	Both parties update the message text
delete_message	Client -> Server	messageId	Delete own message
message_deleted	Server -> Client	messageId	Both parties remove the message
user_status	Server -> All	userId, online, lastSeen	Emitted on every connect / disconnect

6. File Structure

Path	Purpose
Code App/	-- project root
server.js	Express API + Socket.io + Logger (all backend logic)
logger.js	Error logging module -- writes timestamped entries to error.log
error.log	Auto-generated log file (created at runtime)
database.js	Supabase client setup -- exports @supabase/supabase-js instance
db.json	Legacy JSON database (archived -- Supabase now active)
generate_docs.js	This script -- re-run to regenerate this PDF
package.json	npm dependency manifest
public/	-- static frontend files served by Express
index.html	Login, Register, Forgot Password

Path	Purpose
profile.html	Profile editing + Change Password
browse.html	Discover member grid
chat.html	Real-time chat UI
admin.html	Admin dashboard
logs.html	Error log viewer (admin-only, auto-refresh)
documentation.html	Full technical documentation (this page)
terms.html	Terms & Conditions
privacy.html	Privacy Policy
faq.html	FAQ & Troubleshooting
changelog.html	Version history & changelog
nav.js	Shared navbar chip (name + avatar)
global.js	Persistent socket, notifications, tab-title flash
style.css	All styles for all pages
uploads/	Profile photos + chat files (exclude from git)

7. Credentials & Configuration

Setting	Value	Where to change
Server port	3000	server.js -- const PORT
JWT secret	social-app-secret-key-change-in-prod	server.js -- const JWT_SECRET
JWT expiry	7 days	server.js -- jwt.sign expiresIn
Admin password	admin1234	server.js -- const ADMIN_PASSWORD
Profile photo limit	5 MB	server.js -- upload multer config

Setting	Value	Where to change
Chat file limit	10 MB	server.js -- uploadChat multer config
Database file	db.json	database.js -- FileSync path

Security note: Change `JWT_SECRET` and `ADMIN_PASSWORD` before deploying to any public server.

8. Data Models (db.json)

Field	Type	Description
id	number	Auto-incrementing primary key
username	string	Unique; chosen at registration
email	string	Unique email address
password	string	bcrypt hash -- never returned by any API route
name	string null	Display name set on Profile page
age	number null	Age set on Profile page
gender	string null	Gender set on Profile page
bio	string null	Short bio, max 300 characters
photo	string null	Server path, e.g. /uploads/photo.jpg
created_at	ISO string	Registration timestamp
lastSeen	ISO string	Updated on socket disconnect

Field	Type	Description
id	number	Auto-incrementing primary key
sender_id	number	ID of the sending user
receiver_id	number	ID of the recipient

Field	Type	Description
content	string	Message text or /uploads/... path for files
type	"text" "image" "file"	Defaults to "text"
fileName	string null	Original filename for attachments
edited	boolean	true if the sender edited this message
created_at	ISO string	Send timestamp (server time)

9. Reasonix Setup

Reasonix is a DeepSeek-native coding agent that runs in the terminal. It uses DeepSeek's API directly - cache-first loop, flash-first cost control, and automatic tool-call repair.

- Prerequisites**
- Node.js 20.10+ - verify with `node --version`
 - DeepSeek API key - obtain from platform.deepseek.com
 - Git for Windows (Windows users only)

- Step-by-Step Setup**
1. Open Command Prompt - Win + R, type `cmd`, press Enter
 2. Navigate to project folder: `cd C:\Users\andre\Documents\Deepseek\Code App`
 3. Run Reasonix: `npx reasonix@latest code` (first run prompts for API key)
 4. Enter your API key when prompted - saved to `C:\Users\andre\reasonix\config.json`
 5. (Optional) Customise config: `notepad C:\Users\andre\reasonix\config.json`

Cost Comparison

Model	Cost / Turn	Turns for \$4.98	Best For
Flash (default)	~\$0.0006	~8,000	Quick edits, exploration
Pro (/preset max)	~\$0.01–0.03	~200	Complex reasoning tasks

Key Commands

Shortcut / Command	Action
Ctrl + L	Clear terminal
Ctrl + C	Exit Reasonix
Shift + Tab	Auto-apply all suggestions
y / n	Accept or discard a change
/preset max	Switch to DeepSeek Pro model
/stats	Display token / cost statistics
/cost	Show running cost for the session
/clear	Clear the conversation context
/help	Show all available commands

Tip: Use Flash for quick exploration; switch to Pro with `/preset max` for deep reasoning.

Quick-Start Cheat Sheet

```
cd C:\Users\andre\Documents\Deepseek\Code App
npx reasonix@latest code
/preset max
/stats
```

10. Supabase Local Setup

Spark has migrated from lowdb (JSON file) to Supabase (PostgreSQL) for better scalability and Vercel deployment readiness.

- Prerequisites**
- Docker Desktop -- install from [docker.com](https://www.docker.com)
 - Supabase CLI -- installed via npx (no global install needed)

Local Setup

```
cd C:\Users\andre\Documents\Deepseek\Code App
npx supabase init
npx supabase start
```

This spins up PostgreSQL + Supabase Studio + all supporting services locally via Docker.

Local URLs

Service	URL
Supabase Studio (web UI)	http://127.0.0.1:54323
API Endpoint	http://127.0.0.1:54321
Database URL	postgresql://postgres:postgres@127.0.0.1:54322/postgres
Mailpit (test emails)	http://127.0.0.1:54324

Cloud Supabase Spark uses a cloud Supabase project (CWD) at <https://hpezaqvtfuvvczyixwc.supabase.co> for production data. The local Supabase instance is used for development and testing.

- Migration Complete**
1. Phase 1 -- Set up Supabase locally
 2. Phase 2 -- Create tables (users, messages, contacts)
 3. Phase 3 -- Migrated 5 users, 19 messages, 2 contacts from db.json
 4. Phase 4 -- Replaced all lowdb queries with Supabase
 5. Phase 5 -- File uploads moved to Supabase Storage
 6. Phase 6 -- Deploy to Vercel -- live at spark-social-app-ys49.vercel.app